CBIT 3rd CSE-5 — Internal Lab Exam-2 Question Bank & Sample Input/Output (50 Sets)

Strictly Aligned to CBIT CSE Syllabus: UNIT-III (Exception Handling, Multithreading & Generics) & UNIT-IV (Collections, Iterators, Comparator & Java 8 Lambda/Functional Interfaces)

Project Developer & Chief Exam Coordinator: Dr. I. Srujana, Asst. Prof., Dept. of CSE, Chaitanya Bharathi Institute of Technology (A)

Set #	Part A — UNIT-IV (Collections, Iterators, Comparator, Lambda)	Part B — UNIT-III (Exception Handling, Multithreading, Generics)
Set #01	**Part A (Unit-IV: ArrayList):** Develop a Java program to manage a student list using ArrayList, allowing add, remove, and search by roll number. SAMPLE INPUT: 3 101 Rahul 102 Sneha 103 Kiran 102 103 SAMPLE OUTPUT: 101:Rahul 103:Kiran Found: Kiran	**Part B (Unit-III: Exception Handling):** Implement exception handling using try, catch, and throw to validate student data input (age between 17 and 30, marks between 0 and 100). SAMPLE INPUT: Rahul 19 88 SAMPLE OUTPUT: VALID STUDENT: Rahul (Age: 19, Marks: 88)
Set #02	**Part A (Unit-IV: HashMap & Iterator):** Write a Java program to store employee records in a HashMap (EmployeeID → Name) and display them using an Iterator. SAMPLE INPUT: 3 102 Anita 101 Vikram 103 Suresh SAMPLE OUTPUT: 101 -> Vikram 102 -> Anita 103 -> Suresh	**Part B (Unit-III: Multithreading):** Create a worker Thread that prints employee performance scores concurrently with thread execution messages. SAMPLE INPUT: 2 Vikram 92 Anita 88 SAMPLE OUTPUT: Thread evaluating Vikram: Score = 92 Thread evaluating Anita: Score = 88 All Employee Threads Completed
Set #03	**Part A (Unit-IV: TreeSet):** Use a TreeSet to maintain a sorted collection of unique city names entered by the user. SAMPLE INPUT: 4 Hyderabad Chennai Bengaluru Hyderabad SAMPLE OUTPUT: Bengaluru Chennai Hyderabad	**Part B (Unit-III: Generics):** Implement a Generic class Box<T> and a generic method to store and display city pin codes (Integer) and city names (String). SAMPLE INPUT: Hyderabad 500075 SAMPLE OUTPUT: Generic String Box: Hyderabad Generic Integer Box: 500075
Set #04	**Part A (Unit-IV: LinkedList Queue):** Design a Java program that uses LinkedList to simulate a FIFO queue of customers in a bank (enqueue N customers and serve/dequeue K customers). SAMPLE INPUT: 4 2 Ramesh Suresh Mahesh Ganesh SAMPLE OUTPUT: Served: Ramesh, Suresh Waiting in Queue: Mahesh, Ganesh	**Part B (Unit-III: Thread & sleep()):** Create a Thread using Thread.sleep() that prints each country name in order with a simulated delay. SAMPLE INPUT: 3 India Japan Germany SAMPLE OUTPUT: Printing Country: India Printing Country: Japan Printing Country: Germany Thread Execution Finished
Set #05	**Part A (Unit-IV: TreeMap):** Use TreeMap to store product details (ProductID → ProductName) and display them in ascending order of ProductID. SAMPLE INPUT: 3 105 Laptop 101 Mouse 103 Keyboard SAMPLE OUTPUT: 101: Mouse 103: Keyboard 105: Laptop	**Part B (Unit-III: Generics & Exception Handling):** Write a Generic method displayArray(T[] elements) and use exception handling to catch ArrayIndexOutOfBoundsException when querying an index. SAMPLE INPUT: 3 Laptop Mouse Keyboard 1 SAMPLE OUTPUT: Generic Array: Laptop Mouse Keyboard Item at index 1: Mouse

Set #	Part A — UNIT-IV (Collections, Iterators, Comparator, Lambda)	Part B — UNIT-III (Exception Handling, Multithreading, Generics)
Set #06	**Part A (Unit-IV: HashSet):** Create a HashSet to eliminate duplicate book titles from user input while preserving unique titles. **SAMPLE INPUT:** 4 Java_Complete_Reference Effective_Java Java_Complete_Reference Clean_Code **SAMPLE OUTPUT:** **Unique Count: 3** **Clean_Code** **Effective_Java** **Java_Complete_Reference**	**Part B (Unit-III: Thread Priorities):** Demonstrate thread priorities (MIN_PRIORITY, NORM_PRIORITY, MAX_PRIORITY) using three threads processing library tasks. **SAMPLE INPUT:** ArchiveCatalog UpdateIndex IssueBook **SAMPLE OUTPUT:** **MAX_PRIORITY(10):** **IssueBook** **NORM_PRIORITY(5):** **UpdateIndex** **MIN_PRIORITY(1):** **ArchiveCatalog**
Set #07	**Part A (Unit-IV: Comparator):** Implement a Comparator to sort a list of students by their percentage in descending order. **SAMPLE INPUT:** 3 Rahul 82 Sneha 94 Kiran 88 **SAMPLE OUTPUT:** **Sneha: 94%** **Kiran: 88%** **Rahul: 82%**	**Part B (Unit-III: try-catch-finally):** Use a try-catch-finally block to handle division-by-zero (ArithmeticException) during average marks calculation. **SAMPLE INPUT:** 450 5 **SAMPLE OUTPUT:** **Average: 90** **Finally Block Executed**
Set #08	**Part A (Unit-IV: ListIterator):** Develop a ListIterator program to navigate forward and backward through a playlist of songs. **SAMPLE INPUT:** 3 Believer Thunder Radioactive **SAMPLE OUTPUT:** **Forward: Believer Thunder Radioactive** **Backward: Radioactive Thunder Believer**	**Part B (Unit-III: Runnable & Multithreading):** Create multiple threads implementing Runnable to simulate songs playing concurrently. **SAMPLE INPUT:** 2 Believer Thunder **SAMPLE OUTPUT:** **Playing Thread: Believer** **Playing Thread: Thunder** **Playlist Threads Complete**
Set #09	**Part A (Unit-IV: HashMap):** Write a program using HashMap to represent inventory items with product name and quantity, and calculate total stock. **SAMPLE INPUT:** 3 Mouse 25 Keyboard 15 Monitor 10 **SAMPLE OUTPUT:** **Keyboard = 15** **Monitor = 10** **Mouse = 25** **Total Stock = 50**	**Part B (Unit-III: throw & throws Custom Exception):** Implement custom exception OutOfStockException using throw and throws when requested order quantity exceeds available stock. **SAMPLE INPUT:** Laptop 10 4 **SAMPLE OUTPUT:** **Order Confirmed: Laptop, Remaining = 6**
Set #10	**Part A (Unit-IV: ArrayList & Lambda):** Use ArrayList to store and sort bank account balances in ascending order using a Java 8 lambda expression. **SAMPLE INPUT:** 4 45000 12000 78000 30000 **SAMPLE OUTPUT:** **12000 30000 45000 78000**	**Part B (Unit-III: Thread Synchronization):** Demonstrate synchronized bank account withdrawal where two threads attempt to withdraw money from a shared balance. **SAMPLE INPUT:** 1000 600 300 **SAMPLE OUTPUT:** **Withdrawal of 600 successful, Balance = 400** **Withdrawal of 300 successful, Balance = 100**
Set #11	**Part A (Unit-IV: Set Interface):** Use a Set (HashSet/TreeSet) to maintain a unique list of registered voter IDs and display them in sorted order. **SAMPLE INPUT:** 4 VOTER102 VOTER101 VOTER102 VOTER103 **SAMPLE OUTPUT:** **Total Unique Voters: 3** **VOTER101** **VOTER102** **VOTER103**	**Part B (Unit-III: Custom Exception Handling):** Write a program to validate voter age using custom InvalidAgeException (age must be >= 18) with try, catch, and throw. **SAMPLE INPUT:** Kiran 21 **SAMPLE OUTPUT:** **Eligible to Vote: Kiran**

Set #	Part A — UNIT-IV (Collections, Iterators, Comparator, Lambda)	Part B — UNIT-III (Exception Handling, Multithreading, Generics)

Set #12	**Part A (Unit-IV: TreeSet):** Implement a TreeSet to keep a sorted list of unique book authors entered by the user.
	SAMPLE INPUT:
	`4`
	`Schildt`
	`Bloch`
	`Gosling`
	`Bloch`
	SAMPLE OUTPUT:
	`Bloch`
	`Gosling`
	`Schildt`
		Part B (Unit-III: Thread Synchronization): Demonstrate thread synchronization where two threads access and borrow books from the same shared library stock.
		SAMPLE INPUT:
		`JavaBook 5 3 2`
		SAMPLE OUTPUT:
		`Thread-1 borrowed 3, Left = 2`
		`Thread-2 borrowed 2, Left = 0`

Set #13	**Part A (Unit-IV: Map Interface):** Write a program using a Map to store student grades by subject and display them in alphabetical order of subject.
	SAMPLE INPUT:
	`3`
	`Java A+`
	`DBMS A`
	`Algorithms O`
	SAMPLE OUTPUT:
	`Algorithms -> O`
	`DBMS -> A`
	`Java -> A+`
		Part B (Unit-III: Generics & Exception Handling): Write a Generic class Pair<K, V> to store Subject Code and Marks, and validate marks using try-catch for NumberFormatException.
		SAMPLE INPUT:
		`CS301 95`
		SAMPLE OUTPUT:
		`Pair<String, Integer>: CS301 = 95`

Set #14	**Part A (Unit-IV: LinkedList):** Use LinkedList to simulate train bogies attachment (addFirst, addLast) and removal (removeLast).
	SAMPLE INPUT:
	`2`
	`B1 B2`
	`ENG GRD`
	SAMPLE OUTPUT:
	`After Attachment: ENG B1 B2 GRD`
	`After Removing Last: ENG B1 B2`
		Part B (Unit-III: Life Cycle of Thread): Demonstrate Thread life cycle states (NEW, RUNNABLE, TERMINATED) for a train departure monitoring thread.
		SAMPLE INPUT:
		`Falaknuma_Express`
		SAMPLE OUTPUT:
		`Train Falaknuma_Express Thread State after creation: NEW`
		`Train Falaknuma_Express Thread State after start(): RUNNABLE`
		`Train Falaknuma_Express Thread State after completion: TERMINATED`

Set #15	**Part A (Unit-IV: Functional Interface & Lambda):** Demonstrate the use of a Functional Interface with a Lambda Expression to calculate the square of each number in a list.
	SAMPLE INPUT:
	`4`
	`3 5 7 9`
	SAMPLE OUTPUT:
	`9 25 49 81`
		Part B (Unit-III: Generics & Type Parameters): Implement a Generic method countGreaterThan(T[] arr, T elem) using bounded type parameter <T extends Comparable<T>>.
		SAMPLE INPUT:
		`5 20`
		`10 25 30 15 40`
		SAMPLE OUTPUT:
		`Count greater than 20: 3`

Set #16	**Part A (Unit-IV: HashMap):** Create a HashMap to track login attempts per user (count frequency of each username).
	SAMPLE INPUT:
	`5`
	`rahul`
	`sneha`
	`rahul`
	`kiran`
	`rahul`
	SAMPLE OUTPUT:
	`kiran: 1 attempts`
	`rahul: 3 attempts`
	`sneha: 1 attempts`
		Part B (Unit-III: Exception Handling (throw/catch)): Throw and catch a custom AccountLockedException when a user's failed login attempts exceed 3.
		SAMPLE INPUT:
		`rahul 2`
		SAMPLE OUTPUT:
		`Login Allowed for rahul (Attempts: 2)`

Set #17	**Part A (Unit-IV: ArrayList & Collections):** Write a program using ArrayList to store employee salaries and find both the maximum and minimum salary.
	SAMPLE INPUT:
	`4`
	`45000 72000 58000 64000`
	SAMPLE OUTPUT:
	`Max Salary: 72000`
	`Min Salary: 45000`
		Part B (Unit-III: Generics (Bounded Type Parameters)): Implement a Generic class SalaryRecord<T extends Number> that computes the annual salary (12 * monthly) and bonus (10%).
		SAMPLE INPUT:
		`Rahul 50000`
		SAMPLE OUTPUT:
		`Rahul -> Annual: 600000, Bonus: 5000`

Set #	Part A — UNIT-IV (Collections, Iterators, Comparator, Lambda)	Part B — UNIT-III (Exception Handling, Multithreading, Generics)

Set #18	**Part A (Unit-IV: TreeMap):** Create a TreeMap to store university department codes and department names, displaying them alphabetically by code.
	SAMPLE INPUT:
	`3`
	`IT Information_Technology`
	`CSE Computer_Science`
	`ECE Electronics`
	SAMPLE OUTPUT:
	`CSE -> Computer_Science`
	`ECE -> Electronics`
	`IT -> Information_Technology`
		Part B (Unit-III: Thread Sleep & Interruption): Implement thread sleep and interruption (InterruptedException) for parallel department updates.
		SAMPLE INPUT:
		`CSE true`
		SAMPLE OUTPUT:
		`Department CSE: Thread Interrupted & Handled`

Set #19	**Part A (Unit-IV: HashSet):** Use HashSet to store unique course codes offered in a semester and verify if a queried course code is offered.
	SAMPLE INPUT:
	`4`
	`22CSC11 22CSC12 22CSC11 22MTC05`
	`22CSC12`
	SAMPLE OUTPUT:
	`Unique Courses: 3`
	`Course 22CSC12: Offered`
		Part B (Unit-III: Generics & Exception Handling): Implement a Generic method swapElements(T[] arr, int i, int j) and handle ArrayIndexOutOfBoundsException if indices are invalid.
		SAMPLE INPUT:
		`3`
		`CS101 CS102 CS103`
		`0 2`
		SAMPLE OUTPUT:
		`After Swap: CS103 CS102 CS101`

Set #20	**Part A (Unit-IV: List & Lambda Expressions):** Use a List to store and sort student exam scores in ascending order using Java 8 lambda expressions.
	SAMPLE INPUT:
	`5`
	`78 92 65 88 71`
	SAMPLE OUTPUT:
	`Sorted Scores: 65 71 78 88 92`
		Part B (Unit-III: Custom Exception (throw/throws)): Throw and catch a custom LowScoreException when a student's score is below the passing threshold of 40.
		SAMPLE INPUT:
		`Rahul 75`
		SAMPLE OUTPUT:
		`PASS: Rahul scored 75`

Set #21	**Part A (Unit-IV: Map Interface):** Design a Map (HashMap/TreeMap) to store Employee ID and Designation, and look up an employee's designation by ID.
	SAMPLE INPUT:
	`3`
	`E101 Developer`
	`E102 Manager`
	`E103 Analyst`
	`E102`
	SAMPLE OUTPUT:
	`Total Employees: 3`
	`Designation of E102: Manager`
		Part B (Unit-III: Generic Class & Type Parameters): Create a Generic class KeyValue<K, V> to store Employee ID and Department, and demonstrate type safety.
		SAMPLE INPUT:
		`101 CSE_Department`
		SAMPLE OUTPUT:
		`KeyValue<Integer, String>: Key = 101, Value = CSE_Department`

Set #22	**Part A (Unit-IV: Comparator):** Implement Comparator to sort students by GPA in descending order (and alphabetically by name on tie).
	SAMPLE INPUT:
	`3`
	`Rahul 8.5`
	`Sneha 9.4`
	`Kiran 8.9`
	SAMPLE OUTPUT:
	`Sneha - 9.4`
	`Kiran - 8.9`
	`Rahul - 8.5`
		Part B (Unit-III: NullPointerException Handling): Catch and handle NullPointerException during GPA calculation when a student's grade record is null/MISSING.
		SAMPLE INPUT:
		`Rahul 9.2`
		SAMPLE OUTPUT:
		`Verified GPA for Rahul: 9.2`

Set #23	**Part A (Unit-IV: TreeSet):** Create a TreeSet for event participant names and display them in alphabetical order along with first and last participant.
	SAMPLE INPUT:
	`4`
	`Zoya`
	`Aarav`
	`Meera`
	`Kiran`
	SAMPLE OUTPUT:
	`Sorted Participants: Aarav, Kiran, Meera, Zoya`
	`First: Aarav, Last: Zoya`
		Part B (Unit-III: Multithreading & Synchronization): Demonstrate multithreading where two threads register participants into a shared synchronized counter.
		SAMPLE INPUT:
		`25 30`
		SAMPLE OUTPUT:
		`Thread-A registered: 25`
		`Thread-B registered: 30`
		`Synchronized Total Participants: 55`

Set #	Part A — UNIT-IV (Collections, Iterators, Comparator, Lambda)	Part B — UNIT-III (Exception Handling, Multithreading, Generics)

Set #24	**Part A (Unit-IV: LinkedList):** Write a program using LinkedList to simulate a bus ticket booking waiting list (add passengers and remove cancelled passenger).
	SAMPLE INPUT:
	`4`
	`Ravi Sita Gita Mohan`
	`Sita`
	SAMPLE OUTPUT:
	`Active Bus List: Ravi Gita Mohan`
		Part B (Unit-III: Thread Synchronization): Create synchronized threads to simulate multiple passengers booking seats from a limited bus seat pool.
		SAMPLE INPUT:
		`10 3`
		`Ravi 4`
		`Sita 4`
		`Mohan 4`
		SAMPLE OUTPUT:
		`Ravi booked 4 seats (Remaining: 6)`
		`Sita booked 4 seats (Remaining: 2)`
		`Mohan booking failed: Only 2 seats left`

Set #25	**Part A (Unit-IV: Lambda Expression & Functional Interface):** Demonstrate the use of a Java 8 lambda expression (Predicate) to filter even numbers from a list of integers.
	SAMPLE INPUT:
	`6`
	`12 7 18 25 30 9`
	SAMPLE OUTPUT:
	`12 18 30`
		Part B (Unit-III: Multi-catch Exception Handling): Write exception handling using try and multiple catch blocks to handle InputMismatchException / NumberFormatException and NegativeNumberException.
		SAMPLE INPUT:
		`42`
		SAMPLE OUTPUT:
		`Valid Positive Integer: 42`

Set #26	**Part A (Unit-IV: HashMap):** Create a HashMap to store bank customer account numbers and customer names, and display all entries sorted by account number.
	SAMPLE INPUT:
	`3`
	`9002 Sneha`
	`9001 Rahul`
	`9003 Kiran`
	SAMPLE OUTPUT:
	`Account 9001: Rahul`
	`Account 9002: Sneha`
	`Account 9003: Kiran`
		Part B (Unit-III: Custom Exception Handling): Handle exceptions when validating bank account transactions (MinimumBalanceException if balance drops below 1000).
		SAMPLE INPUT:
		`9001 5000 3500`
		SAMPLE OUTPUT:
		`Transaction Success for 9001: New Balance = 1500`

Set #27	**Part A (Unit-IV: ArrayList & Lambda Expressions):** Use ArrayList and Java 8 lambda expressions to calculate the total bill and discounted bill (10% off if total >= 1000) from item prices.
	SAMPLE INPUT:
	`4`
	`250 400 150 200`
	SAMPLE OUTPUT:
	`Total Bill: 1000`
	`Final Payable: 900`
		Part B (Unit-III: try-catch-finally): Use try-catch-finally to validate item prices (reject negative prices) and ensure resource cleanup in the finally block.
		SAMPLE INPUT:
		`3`
		`100 250 150`
		SAMPLE OUTPUT:
		`Valid Bill Sum: 500`
		`Billing Resource Closed in Finally`

Set #28	**Part A (Unit-IV: TreeMap):** Write a TreeMap program to store Department → EmployeeName and display departments sorted alphabetically.
	SAMPLE INPUT:
	`3`
	`Sales Vikram`
	`HR Anita`
	`Engineering Rahul`
	SAMPLE OUTPUT:
	`Engineering: Rahul`
	`HR: Anita`
	`Sales: Vikram`
		Part B (Unit-III: Multithreading): Use multiple threads to calculate department-wise integer salary averages concurrently.
		SAMPLE INPUT:
		`2`
		`CSE 60000 75000 90000`
		`ECE 50000 60000 70000`
		SAMPLE OUTPUT:
		`Thread [CSE] Average Salary = 75000`
		`Thread [ECE] Average Salary = 60000`

Set #29	**Part A (Unit-IV: HashSet):** Create a HashSet to store unique official email addresses from user input and display them sorted alphabetically.
	SAMPLE INPUT:
	`4`
	`rahul@cbit.ac.in`
	`sneha@cbit.ac.in`
	`rahul@cbit.ac.in`
	`kiran@cbit.ac.in`
	SAMPLE OUTPUT:
	`Unique Emails: 3`
	`kiran@cbit.ac.in`
	`rahul@cbit.ac.in`
	`sneha@cbit.ac.in`
		Part B (Unit-III: Custom Exception Handling): Handle invalid email format using a custom InvalidEmailException (must contain '@' and end with '.in' or '.com').
		SAMPLE INPUT:
		`student@cbit.ac.in`
		SAMPLE OUTPUT:
		`Valid Email: student@cbit.ac.in`

Set #	Part A — UNIT-IV (Collections, Iterators, Comparator, Lambda)	Part B — UNIT-III (Exception Handling, Multithreading, Generics)
Set #30	**Part A (Unit-IV: ListIterator):** Implement a ListIterator for a movie rental catalog to traverse forward and replace a discontinued movie title. SAMPLE INPUT: 3 Inception Interstellar Matrix Matrix Avatar SAMPLE OUTPUT: Updated Catalog (Forward): Inception Interstellar Avatar Updated Catalog (Backward): Avatar Interstellar Inception	**Part B (Unit-III: Generic Class):** Demonstrate a Generic class RentalQueue<T> that stores and processes rented items of any type. SAMPLE INPUT: 3 Inception Interstellar Avatar SAMPLE OUTPUT: Generic RentalQueue (3 items): Inception, Interstellar, Avatar
Set #31	**Part A (Unit-IV: LinkedList):** Use LinkedList to store IT service requests in a company (add normal ticket at end, urgent ticket at front). SAMPLE INPUT: 3 TKT101 TKT102 TKT103 URG999 SAMPLE OUTPUT: Priority Service Queue: URG999 TKT101 TKT102 TKT103	**Part B (Unit-III: Thread Interruption):** Handle interrupted threads (InterruptedException) when simulating service response timeouts. SAMPLE INPUT: TKT101 120 500 SAMPLE OUTPUT: Ticket TKT101: Resolved in 120ms
Set #32	**Part A (Unit-IV: Map / TreeMap):** Create a Map (TreeMap) for airport IATA codes and city names, displaying all mappings sorted by IATA code. SAMPLE INPUT: 3 HYD Hyderabad BLR Bengaluru DEL Delhi SAMPLE OUTPUT: BLR -> Bengaluru DEL -> Delhi HYD -> Hyderabad	**Part B (Unit-III: Generics & Custom Exception):** Implement a Generic method findValueByKey(Map<K, V> map, K key) that throws a custom KeyNotFoundException if the airport code is missing. SAMPLE INPUT: 2 HYD Hyderabad BLR Bengaluru HYD SAMPLE OUTPUT: Airport HYD: Hyderabad
Set #33	**Part A (Unit-IV: Comparator):** Implement a Comparator to sort cricket players by match score in descending order. SAMPLE INPUT: 3 Kohli 89 Rohit 104 Gill 72 SAMPLE OUTPUT: Rohit: 104 Kohli: 89 Gill: 72	**Part B (Unit-III: Exception Handling (try-catch)):** Use try-catch to handle invalid player data (NegativeScoreException or NumberFormatException). SAMPLE INPUT: Kohli 95 SAMPLE OUTPUT: Valid Player Record: Kohli (95)
Set #34	**Part A (Unit-IV: TreeSet):** Use TreeSet to store unique customer IDs and display them in ascending sorted order. SAMPLE INPUT: 5 1045 1012 1045 1089 1023 SAMPLE OUTPUT: 1012 1023 1045 1089	**Part B (Unit-III: Thread Synchronization):** Demonstrate synchronized access to shared customer loyalty points updated by two concurrent threads. SAMPLE INPUT: 100 50 75 SAMPLE OUTPUT: After Thread-1 (+50): 150 After Thread-2 (+75): 225
Set #35	**Part A (Unit-IV: List & Iterator):** Use a List (ArrayList) to store names of books and print them with their 1-based position using an Iterator. SAMPLE INPUT: 3 Java_Programming Operating_Systems Database_Systems SAMPLE OUTPUT: 1. Java_Programming 2. Operating_Systems 3. Database_Systems	**Part B (Unit-III: Custom Exception & finally):** Handle custom BookNotFoundException using try-catch-finally when searching for a book in the catalog. SAMPLE INPUT: 2 Java_Programming Operating_Systems Operating_Systems SAMPLE OUTPUT: Book Found: Operating_Systems Catalog Search Closed
Set #36	**Part A (Unit-IV: Map Interface):** Create a Map (TreeMap) for student roll number to grade mapping and display in roll number order. SAMPLE INPUT: 3 283 A+ 281 O 282 A SAMPLE OUTPUT: 281: O 282: A 283: A+	**Part B (Unit-III: Generics):** Implement a Generic class GradeBook<T extends Number> that calculates the total and average of N subject marks. SAMPLE INPUT: 4 80 90 85 95 SAMPLE OUTPUT: Total = 350, Average = 87

Set #	Part A — UNIT-IV (Collections, Iterators, Comparator, Lambda)	Part B — UNIT-III (Exception Handling, Multithreading, Generics)

Set #37	**Part A (Unit-IV: HashMap):** Implement HashMap to maintain product stock quantity and update stock when new shipment arrives.
	SAMPLE INPUT:
	`2`
	`Laptop 10`
	`Mouse 20`
	`Laptop 5`
	SAMPLE OUTPUT:
	`Laptop: 15`
	`Mouse: 20`
		Part B (Unit-III: Thread Synchronization): Use synchronized threads to simulate customers buying products simultaneously from shared stock.
		SAMPLE INPUT:
		`Laptop 10 6 3`
		SAMPLE OUTPUT:
		`Customer-1 bought 6 Laptop, Stock Left = 4`
		`Customer-2 bought 3 Laptop, Stock Left = 1`

Set #38	**Part A (Unit-IV: LinkedList Queue):** Write a program using LinkedList for a restaurant order queue (add orders and serve the first order).
	SAMPLE INPUT:
	`3`
	`Biryani PaneerNaan GulabJamun`
	SAMPLE OUTPUT:
	`Now Serving: Biryani`
	`Pending Orders: PaneerNaan GulabJamun`
		Part B (Unit-III: Exception Handling): Handle invalid order quantity or price using try-catch-throw (InvalidOrderException if quantity <= 0).
		SAMPLE INPUT:
		`Biryani 2 250`
		SAMPLE OUTPUT:
		`Order Placed: Biryani x2 = Rs.500`

Set #39	**Part A (Unit-IV: TreeMap):** Use TreeMap to store countries and their capitals, displaying them alphabetically by country name.
	SAMPLE INPUT:
	`3`
	`Japan Tokyo`
	`India NewDelhi`
	`France Paris`
	SAMPLE OUTPUT:
	`France -> Paris`
	`India -> NewDelhi`
	`Japan -> Tokyo`
		Part B (Unit-III: Multithreading & sleep()): Create a Thread that prints each country name with a simulated delay using Thread.sleep().
		SAMPLE INPUT:
		`3`
		`India`
		`Japan`
		`France`
		SAMPLE OUTPUT:
		`Printing Country: India`
		`Printing Country: Japan`
		`Printing Country: France`
		`Thread Execution Finished`

Set #40	**Part A (Unit-IV: HashSet):** Use HashSet to remove duplicate vehicle registration numbers and display unique registrations in sorted order.
	SAMPLE INPUT:
	`4`
	`TS09AB1234`
	`TS08XY5678`
	`TS09AB1234`
	`TS07CD9999`
	SAMPLE OUTPUT:
	`Unique Vehicles: 3`
	`TS07CD9999`
	`TS08XY5678`
	`TS09AB1234`
		Part B (Unit-III: Generics & Custom Exception): Implement a Generic class VehicleRegistry<T> and validate registration format using custom InvalidRegistrationException.
		SAMPLE INPUT:
		`TS09AB1234`
		SAMPLE OUTPUT:
		`Registered in Generic Vault: TS09AB1234`

Set #41	**Part A (Unit-IV: Lambda Expression):** Demonstrate a Java 8 lambda expression and Functional Interface to sum all elements of a list.
	SAMPLE INPUT:
	`4`
	`10 20 30 40`
	SAMPLE OUTPUT:
	`Lambda Sum: 100`
		Part B (Unit-III: Exception Handling (Multiple Catch)): Handle ArithmeticException (division by zero) and ArrayIndexOutOfBoundsException during array division calculation.
		SAMPLE INPUT:
		`3 5 1`
		`20 50 80`
		SAMPLE OUTPUT:
		`Result: 10`

Set #42	**Part A (Unit-IV: Map Interface):** Use Map (TreeMap) to track attendance percentage of students and identify students eligible for exams (>= 75%).
	SAMPLE INPUT:
	`3`
	`Rahul 82`
	`Sneha 68`
	`Kiran 75`
	SAMPLE OUTPUT:
	`Kiran: 75% (ELIGIBLE)`
	`Rahul: 82% (ELIGIBLE)`
	`Sneha: 68% (DETAINED)`
		Part B (Unit-III: Custom Exception (throw/throws)): Throw and catch a custom AttendanceShortageException when a student's attendance is below 75%.
		SAMPLE INPUT:
		`Rahul 85`
		SAMPLE OUTPUT:
		`Hall Ticket Issued for Rahul (85%)`

Set #	Part A — UNIT-IV (Collections, Iterators, Comparator, Lambda)	Part B — UNIT-III (Exception Handling, Multithreading, Generics)
Set #43	**Part A (Unit-IV: Comparator):** Implement Comparator for sorting software projects by deadline day number (ascending). SAMPLE INPUT: 3 CodeShield 15 AI_Chatbot 7 Cloud_Portal 22 SAMPLE OUTPUT: AI_Chatbot (Day 7) CodeShield (Day 15) Cloud_Portal (Day 22)	**Part B (Unit-III: Multithreading & join()):** Create worker threads to simulate parallel project module updates and join() them before final deployment. SAMPLE INPUT: 3 AuthModule GraderEngine ProctorLock SAMPLE OUTPUT: Module Thread Completed: AuthModule Module Thread Completed: GraderEngine Module Thread Completed: ProctorLock All Modules Joined: Deployment Ready
Set #44	**Part A (Unit-IV: Set / TreeSet):** Create a Set (TreeSet) of registered 10-digit phone numbers to eliminate duplicates and display in sorted order. SAMPLE INPUT: 4 9876543210 9123456780 9876543210 9000011111 SAMPLE OUTPUT: Unique Phone Numbers: 3 9000011111 9123456780 9876543210	**Part B (Unit-III: Exception Handling):** Validate phone numbers using exception handling (throw IllegalArgumentException if not exactly 10 digits). SAMPLE INPUT: 9876543210 SAMPLE OUTPUT: Valid Phone: 9876543210
Set #45	**Part A (Unit-IV: ArrayList):** Use ArrayList to maintain shopping cart items (add N items, remove 1 item, display remaining cart). SAMPLE INPUT: 4 Watch Shoes Backpack Sunglasses Shoes SAMPLE OUTPUT: Cart Items (3): Watch Backpack Sunglasses	**Part B (Unit-III: Generics & Exception Handling):** Demonstrate a Generic class CartItem<T, U> for storing item name and price, and handle negative price exceptions. SAMPLE INPUT: Backpack 1499 SAMPLE OUTPUT: CartItem<String, Integer>: Backpack = Rs.1499
Set #46	**Part A (Unit-IV: TreeMap):** Write a TreeMap program to store library members with MembershipID → MemberName in sorted ID order. SAMPLE INPUT: 3 302 Sneha 105 Rahul 210 Kiran SAMPLE OUTPUT: M#105: Rahul M#210: Kiran M#302: Sneha	**Part B (Unit-III: Multithreading & Synchronization):** Use synchronized threads to simulate book issuance to multiple library members. SAMPLE INPUT: 2 Rahul Effective_Java Sneha Clean_Code SAMPLE OUTPUT: Synchronized Issue: Effective_Java issued to Rahul Synchronized Issue: Clean_Code issued to Sneha All Library Transactions Completed
Set #47	**Part A (Unit-IV: HashSet):** Create a HashSet to store unique social media usernames and display them in alphabetical order. SAMPLE INPUT: 4 coder_rahul java_sneha coder_rahul tech_kiran SAMPLE OUTPUT: Unique Handles: 3 @coder_rahul @java_sneha @tech_kiran	**Part B (Unit-III: Exception Handling):** Handle exceptions for invalid username input (must be >= 5 characters and contain no spaces or invalid symbols). SAMPLE INPUT: coder_rahul SAMPLE OUTPUT: Valid Username: @coder_rahul

Set #	Part A — UNIT-IV (Collections, Iterators, Comparator, Lambda)	Part B — UNIT-III (Exception Handling, Multithreading, Generics)
Set #48	**Part A (Unit-IV: ListIterator):** Implement ListIterator to traverse a list of bank transaction IDs forward and backward. **SAMPLE INPUT:** 3 TXN101 TXN102 TXN103 **SAMPLE OUTPUT:** Chronological: TXN101 TXN102 TXN103 Reverse Chronological: TXN103 TXN102 TXN101	**Part B (Unit-III: Generic Methods & Comparable):** Implement a Generic method findMaxTransaction(T[] arr) using <T extends Comparable<T>> to find the highest transaction amount. **SAMPLE INPUT:** 4 12500 48000 32000 19000 **SAMPLE OUTPUT:** Highest Transaction: 48000
Set #49	**Part A (Unit-IV: Lambda Expression & Streams/Filter):** Use a Java 8 lambda expression to filter high-scoring students (marks >= 75) from a list. **SAMPLE INPUT:** 4 Rahul 82 Sneha 68 Kiran 91 Divya 74 **SAMPLE OUTPUT:** Rahul: 82 Kiran: 91	**Part B (Unit-III: Exception Handling):** Handle exceptions during parsing and validation of student marks (NumberFormatException and MarksOutOfRangeException). **SAMPLE INPUT:** Rahul 88 **SAMPLE OUTPUT:** Recorded: Rahul = 88
Set #50	**Part A (Unit-IV: Map Interface):** Write a Map (TreeMap) program to store employee name and salary pairs and display them sorted by employee name. **SAMPLE INPUT:** 3 Vikram 75000 Anita 82000 Rahul 68000 **SAMPLE OUTPUT:** Anita -> Rs.82000 Rahul -> Rs.68000 Vikram -> Rs.75000	**Part B (Unit-III: Thread Synchronization):** Implement thread synchronization while two threads apply consecutive salary increments to an employee's salary. **SAMPLE INPUT:** Vikram 50000 5000 8000 **SAMPLE OUTPUT:** Thread-1 Increment (+5000): Vikram Salary = 55000 Thread-2 Increment (+8000): Vikram Salary = 63000